

Git y GitHub



ACTUMLOGOS

DESARROLLANDO HABILIDADES TECNOLÓGICAS

Módulo 1: Fundamentos de Git



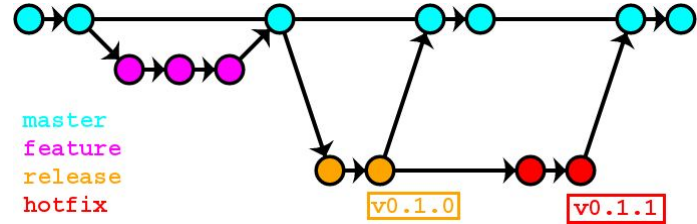
ACTUMLOGOS

DESARROLLANDO HABILIDADES TECNOLÓGICAS

¿Qué es Git?

Git es un **sistema de control de versiones distribuido** que permite:

- Guardar el historial completo de un proyecto.
- Volver a cualquier estado anterior.
- Trabajar en equipo sin sobrescribir trabajo.
- Crear líneas de trabajo independientes (ramas).



¿Cómo funciona Git por dentro?

Git usa tres espacios principales:

- **Working Directory:**
Tus archivos normales, en tu computadora.
- **Staging Area (Área de preparación):**
Cambios *marcados* para ser guardados.
- **Repository (.git):**
Donde se almacena el historial real del proyecto.

Proceso mental:

Editar → *Preparar* → *Guardar definitivamente*

El ciclo básico de Git (muy importante)

git init — Crear un repositorio

- Inicializa un repositorio Git en una carpeta.

git add — Preparar archivos (Staging Area)

- Selecciona qué archivos quieres incluir en la próxima versión.

git commit — Guardar una versión

- Crea un **punto de control** del proyecto.

git init

1. Crea la carpeta oculta **.git/**

Este directorio contiene *toda la lógica interna de Git*, incluyendo:

- **Historial completo** de versiones (commits).
- **Configuraciones** del repositorio.
- **Ramas** (como **main** o **develop**).
- **Área de staging** (donde se preparan los cambios).
- **Referencias y metadatos** necesarios para que Git funcione.

Sin esta carpeta, Git **no existe** en el proyecto.

git add

Comandos

```
git add archivo.txt
```

```
git add .
```

¿Qué hace?

- Mueve cambios desde el **working directory** hacia la **staging area**.
- En otras palabras:
 “Estos archivos SÍ quiero incluirlos en la próxima versión.”

Puntos clave

- La staging area permite *controlar* qué se guardará en el commit.
- No crea versiones todavía; solo prepara los archivos.

git commit

- **Comando:**

```
git commit -m "Mensaje descriptivo"
```

- **¿Qué hace?**

Crea una nueva versión del proyecto (snapshot), solo incluye los cambios en la *staging area* y es el paso donde Git realmente guarda la historia.

- **¿Cómo escribir un buen mensaje?**

Debe responder: **¿Qué cambié y por qué?**

Ejemplos: "Corrige bug en función de login", "Agrega validación de formulario".

git commit (plus)

Comandos más usados:

- **git commit**
Abre el editor para escribir el mensaje manualmente.
- **git commit -a -m "mensaje"**
Incluye automáticamente todos los archivos *modificados y rastreados* (tracked).
⚠ No agrega archivos nuevos.
- **git commit --amend**
Permite modificar el último commit (mensaje o contenido).
- **git commit -amend -m "nuevo mensaje"**
Reemplaza el mensaje del último commit.
- **git commit --no-verify**
Salta validaciones como *pre-commit hooks*.

¿Qué es .gitignore?

Archivo que le dice a Git **qué NO debe rastrear** ni incluir en el repositorio.

Ejemplos comunes:

`venv/` — entornos virtuales

`node_modules/` — dependencias de Node

`.env` — credenciales y variables sensibles

`__pycache__/` — archivos temporales de Python

```
bash Copy code

# Byte-compiled files
__pycache__/
*.py[cod]

# Virtual Environments
venv/
*.venv
.env

# IDE related (e.g., PyCharm)
.idea/

# OS generated files
.DS_Store
Thumbs.db

# Log and databases
*.log
*.sqlite
```

¿Por qué es importante?

Evita subir archivos pesados, temporales o con información sensible y mantiene el repositorio limpio.

Reglas rápidas para entender `.gitignore`

- `carpeta/` → ignora toda la carpeta
- `*.ext` → ignora todos los archivos con esa extensión
- `.env` → ignora solo ese archivo
- Si **ya se subió** antes → `.gitignore` NO lo borra (hay que quitarlo del tracking)

Git status

¿Qué hace?

Muestra el estado del repositorio:

- Qué cambió
- Qué no está guardado
- Qué está listo para commit
- Si el repo está limpio o no (“nothing to commit, working tree clean”)

Por qué es útil:

Es el comando más usado después de `git add` y `git commit`.

Git log

Comandos:

```
git log
```

```
git log --oneline (versión compacta)
```

¿Qué hace?

Muestra el historial del proyecto:

- Todas las versiones creadas
- Quién hizo cada commit
- Cuándo se hicieron
- Mensajes descriptivos

Git diff

Comando:

```
git diff
```

¿Qué hace?

Muestra las diferencias entre el **working directory** y la **staging area**.

Permite ver qué cambiaste *antes* de preparar archivos o hacer commit.

¿Para qué sirve?

Para revisar línea por línea qué modificaste y evitar subir cambios no deseados.

Primer ejercicio práctico



ACTUMLOGOS

DESARROLLANDO HABILIDADES TECNOLÓGICAS

Primer ejercicio práctico

Objetivo: Registrar tus primeros cambios con Git.

Pasos:

1. Crear una carpeta del proyecto
2. Crear un archivo `README.md`
3. Ejecutar:
 - `git init`
 - `git add README.md`
 - `git commit -m "Primer commit del proyecto"`
4. Ver historial con `git log`

Resultado: Tu primer repositorio versionado funcionando.

Fin de módulo

- ✓ Entiendes cómo funciona Git por dentro
- ✓ Dominas el ciclo **init** → **add** → **commit**
- ✓ Sabes usar **.gitignore**
- ✓ Identificar el estado del repositorio con **git status**
- ✓ Puedes crear tus primeros proyectos versionados
- ✓ Comprendes cómo visualizar el historial (**git log**)

Conclusión: Ya tienes las bases para trabajar como un programador profesional.